

Les Tableaux en PHP

- Un tableau php est une collection ordonnée de paires (key, value), où la clé “key” est utilisée pour identifier et accéder à la valeur “value” dans la collection
- la valeur de la clé peut être un entier ou une string
- La valeur associée à une clé est quelconque : entier, flottant, string, tableau ...
- Dans un même tableau, on peut utiliser des valeurs de clés entières ou string et stocker des valeurs de types quelconques.

■ Syntaxe de création :

key	value
1	"un"
2	"deux"
"nom"	"franky"
"age"	42
"genre"	'M'
"vivant"	true
"tel"	"06 01 03 05 33"

```
$frank = [  
    1 => "un",  
    2 => "deux",  
    "nom" => "franky",  
    "age" => 42,  
    "genre" => 'M',  
    "vivant" => true,  
    "tel" => "06 01 03 05 33"  
];
```

```
$frank = array(  
    1 => "un",  
    2 => "deux",  
    "nom" => "franky",  
    "age" => 42,  
    "genre" => 'M',  
    "vivant" => true,  
    "tel" => "06 01 03 05 33"  
);
```

Utilisation du tableau

- accès à un élément : `$t[ clé ]`
- tester l'existence d'un élément : `isset( $t[clé] )`
- ajouter un élément (toujours possible) :
`$t[clé]=valeur`
- Supprimer un élément : `unset( $t[clé] )`

```
$age = $frank[ "age" ];           // accéder à un élément
if ( isset($frank['prenom']) ) {} // tester l'existence
unset($frank[2]);                 // suppression d'un
élément
$frank['prenom'] = 'mikael';      // ajout d'un élément

/* création d'un tableau par ajout des éléments */

$mick = [];                      // tableau vide
$mick['nom'] = 'jagger';
$mick['age'] = 78;
```

- La clé peut être oubliée lors de l'ajout d'une valeur.
 - Choisie automatiquement : plus grande clé entière + 1

```
$t = [ 5 => 1, 12 => 2];

$t[] = 56; // ajout d'un élément de clé 13
           // pareil que $t[13] = 56;

// ce tableau :
$a = [ 5 => 43, 32, 56, "b" => 12 ];

// est le même que ce tableau
$a = [ 5 => 43, 6 => 32, 7 => 56, "b" => 12 ];

/* remplir un tableau : */
$t = [];
for ($i=0; $i < 20; $i++) $t[] = 2**$i;
```

- Un tableau peut stocker tous types de valeurs, y compris un tableau :

```
$repertoire = [  
    'john' => ['name'=>"john doe",  
               'email'=>"j.doe@hotmail.com" ],  
    'sarah' => ['name'=>"sarah Lenti",  
               'email'=>"sl@free.fr" ],  
    'quentin'=> ['name'=>"Quentin Tarantino",  
               'email'=>"qt@kill-bill.fi" ]  
];  
  
//echoing Quentin Tarantino email !!  
echo "mail quentin : {$repertoire['quentin']['email']} \n";
```

- C'est la façon de stocker des tableaux à dimension multiples (ex. des matrices)

Itération sur les tableaux

- Très courant, mais les boucles traditionnelles `for` et `while` ne sont pas applicables :
 - Toutes les clés ne sont pas forcément entières et on ne connaît pas leur valeur
 - Même si elles sont toutes entières, elles ne sont pas toutes forcément présentes, il peut y avoir des trous.

■ Utiliser la boucle spécialisée FOREACH

```
foreach ($arr as $value) statement  
foreach ($arr as $key => $value) statement
```

- *\$key* et *\$value* sont des variables qui vont successivement stocker les éléments du tableau à chaque itération

```
$t = [ 'a', 'b', 'c', 'd', 'e'=>'euuuh', 26=>'z' ];  
foreach ($t as $index => $item) {  
    echo "entrée: $index : $item -- {$t[$index]}";  
}
```

■ Affectation de tableaux

- Affectation directe
- Affectation d'une liste de variables

```
$a = ['r', 'o', 'j', 'v', 'b', 'i', 'v'];  
$tab_colors = $a;  
list($c1, $c2, $c3, $c4, $c5, $c6, $c7) = $a;
```


Quelques fonctions sur les tableaux

```
count(Countable|array $value, int $mode = COUNT_NORMAL): int
```

Compte tous les éléments dans un tableau quand utilisé avec un [tableau](#). Quand utilisé avec un objet qui implémente l'interface [Countable](#), ceci retourne la valeur de la méthode [Countable::count\(\)](#).

```
array_key_exists(string|int $key, array $array): bool
```

array_key_exists() retourne **true** s'il existe une clé du nom de **key** dans le tableau **array**. **key** peut être n'importe quelle valeur valide d'index de tableau.

```
array_merge(array ...$arrays): array
```

array_merge() rassemble les éléments d'un ou de plusieurs tableaux en ajoutant les valeurs de l'un à la fin de l'autre. Le résultat est un tableau.

```
array_push(array &$array, mixed ...$values): int
```

array_push() considère **array** comme une pile, et empile les variables **var**, ... à la fin de **array**. La longueur du tableau **array** augmente d'autant. Cela a le même effet que :

Tableaux et chaines de caractères

- Une chaine peut être manipulée comme un tableau de caractères :

```
$str = "une chaine";  
$prems = $str[0]; $deuze = $str[1];  
$derche = $str[ strlen($str) - 1 ];  
$str[3] = '-';
```

- Convertir une chaine en un tableau :

```
$sp = str_split($str);  
// $sp == ['u','n','e',' ','c','h','a','i','n','e']  
$ex = explode(' ', $str); // $ex == ['une', 'chaine']
```

- Rassembler les éléments d'un tableau dans une chaine :

```
$mum = implode('_', ['a', 'good', 'person']);  
print $mum; // "a_good_person"
```

Types et tableau

- `Array` : le type tableau, à utiliser dans les profils de fonctions lorsque l'on attend ou retourne un tableau
- `iterable` : pseudo-type rassemblant les tableaux et les objets implémentant l'interface `Traversable`.
Permet de désigner les valeurs sur lesquelles on peut utiliser `foreach()`

```
function bar(iterable $it): array {  
    $t=[];  
    foreach ($it as $v) array_push($t,strtoupper($v));  
    return $t;  
}
```

Tableaux Prédéfinis (SuperGlobaux)

- Plusieurs variables sont prédéfinies en PHP et sont "superglobales" : elles sont disponibles quelque soit le contexte.
- Ces variables sont toutes des tableaux, notamment :
 - `$_ENV` : tableau contenant les variables d'environnement passées au script
 - `$_GET` : tableau contenant les valeurs transmises via la query_string de l'URL

- Exemple : utilisation du tableau `$_GET` pour transmettre des arguments à un programme exécuté sur un serveur et désigné par une URL

`http://localhost/exget.php?nom=tarantino&prenom=quentin`

query_string

```
$_GET = [  
    'nom'      => 'tarantino',  
    'prenom' => 'quentin'  
]
```

```
<?php
```

```
Print "Bonjour {$_GET['prenom']} {$_GET['nom']}";
```